W1

INHERITANCE

Data Structures & Object-Oriented Programming



Table of Contents

- ปัญหาของการ Copy-Paste Class
- Is-a Relationship
- Inheritance
- Method Overriding
- Method Resolution Order (MRO)
- Common Mistakes

ปัญหาของการ Copy-Paste Class

- สมมติว่าธนาคารต้องการขยายระบบโดยเพิ่มบัญชีออมทรัพย์ที่มีดอกเบี้ย ถ้าไม่รู้จัก Inheritance เราอาจเขียนแบบนี้

แนวทางที่ผิด — Copy-Paste และแก้ไขบางส่วน

```
class SavingsAccount:
    def __init__(self, owner: str, balance: float = 0.0, interest_rate: float = 0.02):
        self._owner    = owner           # ← เหมือน BankAccount ทุกตัวอักษร
        self.balance   = balance         # ← เหมือน BankAccount ทุกตัวอักษร
        self._interest_rate = interest_rate  # เพิ่มใหม่
```

- ถ้าพบ bug ใน BankAccount.withdraw() ต้องไปแก้ทุก class ที่ copy ไปด้วย
- ถ้าเพิ่มบัญชีประเภทใหม่ (CheckingAccount, FixedDepositAccount) code จะซ้ำกันมากขึ้นเรื่อยๆ
- ไม่มีความสัมพันธ์ทาง type ระหว่าง BankAccount กับ SavingsAccount แม้ทั้งสองจะทำสิ่งเดียวกัน
- เมื่อต้องการเพิ่ม feature ให้กับ "บัญชีทุกประเภท" ต้องแก้หลายที่พร้อมกัน

Inheritance แก้ปัญหาทั้งหมดนี้ด้วยการสร้างความสัมพันธ์แบบ parent-child ระหว่าง class
โดย child class ได้รับ ทุกสิ่งจาก parent โดยอัตโนมัติ และเพิ่มหรือเปลี่ยนแปลงเฉพาะส่วนที่จำเป็น

Is-a Relationship

Is-a Relationship

- ก่อนตัดสินใจให้ class หนึ่ง inherit จากอีก class หนึ่ง ให้ถามคำถามนี้เสมอว่า "ChildClass IS-A ParentClass หรือไม่?"
  - ถ้าคำตอบคือ "ใช่" อย่างสมเหตุสมผลในทุกบริบท Inheritance เหมาะสม
  - ถ้าคำตอบคือ "ไม่" หรือ "บางครั้ง" ควรพิจารณา composition แทน (ซึ่งจะเรียนในสัปดาห์ที่ 6)

ตัวอย่างที่ผ่าน is-a test:	ตัวอย่างที่ไม่ผ่าน:
SavingsAccount IS-A BankAccount	Customer IS-A BankAccount ✗
Dog IS-A Animal	(Customer HAS-A BankAccount)
Car IS-A Vehicle	
Manager IS-A Employee	Engine IS-A Car ✗
	(Car HAS-A Engine)

เมื่อเลือกใช้ Inheritance ในสถานการณ์ที่ไม่เหมาะสม ส่งผลให้ child class มีความผูกพันกับ parent class มากเกินไปและได้รับ interface ที่ไม่ต้องการมาด้วย

Inheritance

Syntax ของ Inheritance

```
class ChildClass(ParentClass):
    def __init__(self, ...):
        super().__init__(...)
        # กำหนด attributes เพิ่มเติม
```

เรียก parent ก่อนเสมอ

- ChildClass เรียกว่า subclass หรือ derived class
- ParentClass เรียกว่า superclass หรือ base class
- ทันทีที่ประกาศแบบนี้ ChildClass จะ ได้รับ attributes และ methods ทั้งหมดจาก ParentClass โดยไม่ต้องเขียนซ้ำ

- ทุก class ใน Python สืบทอดจาก object อยู่แล้ว ดังนั้น class BankAccount: มีความหมายเดียวกับ class BankAccount(object):

super().init()

- super() คือ built-in function ที่คืนค่า proxy object ที่ชี้ไปยัง parent class
- เมื่อ child class ถูกสร้างขึ้น Python ไม่ได้เรียก __init__ ของ parent ให้โดยอัตโนมัติ
  - หากผู้พัฒนาไม่เรียกเอง parent จะไม่มีโอกาสเตรียม attribute ของตัวเองเลย
  - ผลที่ตามมาคือ object ที่สร้างขึ้นมาจะขาด attribute ที่ parent ควรจะกำหนดไว้ตั้งแต่ต้น
  - และเมื่อ method ใดพยายามเข้าถึง attribute เหล่านั้น โปรแกรมจะพังโดยไม่มีสัญญาณเตือนล่วงหน้า

```python
class Employee:
    def __init__(self, name: str, salary: float):
        self.name = name
        self.salary = salary

class Manager(Employee):
    def __init__(self, name: str, salary: float, department: str):
        # ลืมเรียก super().__init__() !
        self.department = department

m = Manager("Somchai", 80000, "Engineering")
print(m.department)   # "Engineering" — ใช้ได้
print(m.name)         # AttributeError: 'Manager' object has no attribute 'name'
```

Manager กำหนดเฉพาะ department แต่ไม่เคยบอกให้ Employee เตรียม name และ salary ดังนั้น attribute สองตัวนี้จึงไม่มีอยู่ใน object แม้ว่า Manager จะ inherit มาจาก Employee ก็ตาม

super().init()

```python
class Employee:
    def __init__(self, name: str, salary: float):
        self.name = name
        self.salary = salary
```

- เมื่อเรียก super().init() อย่างถูกต้อง

```python
class Manager(Employee):
    def __init__(self, name: str, salary: float, department: str):
        super().__init__(name, salary)    # ให้ Employee เตรียม name และ salary ก่อน
        self.department = department      # จากนั้นจึงเพิ่มในส่วนของ Manager

m = Manager("Somchai", 80000, "Engineering")
print(m.name)          # "Somchai"
print(m.salary)        # 80000
print(m.department)    # "Engineering"
```

- ลำดับการทำงานในบรรทัด super().__init__(name, salary)
  - Python จะวิ่งขึ้นไปหา Employee.__init__() และส่ง name กับ salary เข้าไปให้ Employee จัดการ
  - เมื่อ Employee กำหนด attribute ของตัวเองเสร็จแล้ว Python จึงกลับมาทำงานในบรรทัดถัดไปของ Manager ต่อ
    - กำหนด self.department

super().init()

- super() คือ built-in function ที่คืนค่า proxy object ที่ชี้ไปยัง parent class ตาม MRO
- การเรียก super().__init__(...) จึงหมายถึง "เรียก __init__ ของ parent class ด้วย arguments เหล่านี้"

```
SavingsAccount("Alice", 10000.0, 0.05)
      |
      ▼
SavingsAccount.__init__ เริ่มทำงาน
      |
      ├── super().__init__("Alice", 10000.0)
      |         |
      |         └── BankAccount.__init__ ทำงาน
      |               ├── self._owner = "Alice"
      |               └── self.balance = 10000.0  (ผ่าน setter)
      |
      └── self._interest_rate = 0.05  ← เพิ่มหลังจาก parent เสร็จแล้ว
```

การเรียก super().__init__() ก่อนเป็นธรรมเนียมที่บอกว่า "child เคารพสัญญาของ parent ก่อนที่จะเพิ่มส่วนของตัวเอง"

Method Overriding

Method Overriding : เปลี่ยนพฤติกรรมใน Child

- เมื่อ child class นิยาม method ที่มีชื่อเดียวกับ parent
  - Python จะใช้ child's version เสมอ เมื่อเรียกผ่าน object ของ child class นั้น

```
Python ค้นหา method โดยเริ่มจาก class ที่ใกล้ที่สุดก่อน:

  object.display_info()
        |
        ▼
  Python ค้น: SavingsAccount.display_info → พบ! ใช้ version นี้
  (ไม่ค้นต่อไปยัง BankAccount)

  object.deposit()
        |
        ▼
  Python ค้น: SavingsAccount.deposit → ไม่พบ
  Python ค้น: BankAccount.deposit → พบ! ใช้ version นี้
```

Method Overriding : เปลี่ยนพฤติกรรมใน Child

- การ override มีสองแนวทางหลัก
  - Override สมบูรณ์ ซึ่งเขียนใหม่ทั้งหมดโดยไม่เรียก parent
    - เหมาะเมื่อ behavior ต้องเปลี่ยนแปลงโดยสิ้นเชิง
  - Override แบบขยาย ซึ่งเรียก super() เพื่อดึง behavior เดิมมาก่อนแล้วเพิ่มส่วนใหม่
    - เหมาะเมื่อต้องการ behavior เดิมบวกส่วนที่เพิ่มขึ้น

Override สมบูรณ์ — เขียนใหม่ทั้งหมด

```
class SavingsAccount(BankAccount):
    def display_info(self) -> None:
        print(f"[SavingsAccount] {self.owner} | {self.balance:.2f} บาท | ดอกเบี้ย:
{self._interest_rate*100:.1f}%")
```

Override แบบขยาย — เพิ่มต่อจาก parent

```
class SavingsAccount(BankAccount):
    def display_info(self) -> None:
        super().display_info()   # แสดงข้อมูลพื้นฐานจาก BankAccount ก่อน
        print(f"  อัตราดอกเบี้ย: {self._interest_rate * 100:.1f}%")   # แล้วเพิ่มส่วนของตัวเอง
```

Multiple Inheritance

- Python รองรับการ inherit จากหลาย parent พร้อมกัน:

```
class D(B, C):
    pass
```

D รับ features จากทั้ง B และ C

กรณีนี้เรียกว่า Diamond Problem เมื่อทั้ง B และ C มี method ชื่อเดียวกัน Python ต้องตัดสินว่าจะใช้ version ของใคร กลไกที่แก้ปัญหานี้คือ MRO

Method Resolution Order (MRO)

Method Resolution Order (MRO)

- MRO คือ ลำดับที่ Python ค้นหา method เมื่อเรียกใช้งาน
- Python ใช้อัลกอริทึม C3 Linearization เพื่อคำนวณ MRO ซึ่งรับประกันว่า child class จะมาก่อน parent และลำดับที่ระบุใน class definition จะถูกเคารพ

```python
class A:
    def greet(self) -> str:
        return "จาก A"

class B(A):
    def greet(self) -> str:
        return "จาก B"

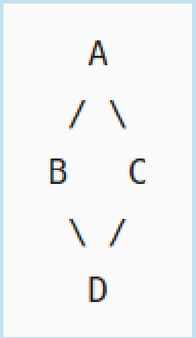
class C(A):
    def greet(self) -> str:
        return "จาก C"

class D(B, C):    # ระบุ B ก่อน C
    pass

d = D()
print(d.greet())      # "จาก B"
print(D.__mro__)
```

(<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)

Diamond Problem — โครงสร้างที่ทำให้เกิดความคลุมเครือ:



MRO ของ D: D → B → C → A → object

Python ค้น method จาก D ก่อน
ถ้าไม่พบไปที่ B ถ้าไม่พบไปที่ C ถ้าไม่พบไปที่ A ตามลำดับ เมื่อพบที่ใดก็หยุดค้นทันที
ลำดับนี้ทำให้ D ค้น B ก่อน C เสมอ เพราะระบุ class D(B, C)

ในทางปฏิบัติ multiple inheritance ควรใช้อย่างระมัดระวัง

ฝึกเขียนโปรแกรม



Live Coding: BankAccount → SavingsAccount

- โจทย์: ระบบธนาคารต้องการรองรับ "บัญชีออมทรัพย์" ซึ่งมีคุณสมบัติทุกอย่างเหมือนบัญชีทั่วไป
  - แต่มีสิ่งพิเศษเพิ่มเติมคือการคิดดอกเบี้ยและข้อจำกัดการถอนเงิน
  - แทนที่จะเขียน class ใหม่ทั้งหมด เราจะใช้ Inheritance เพื่อต่อยอดจาก BankAccount ที่มีอยู่แล้ว

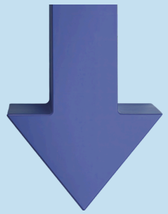
- สิ่งที่ SavingsAccount มีเพิ่มจาก BankAccount
  - SavingsAccount รับ interest_rate (อัตราดอกเบี้ยต่อปีในรูปทศนิยม เช่น 0.03 หมายถึง 3%) เป็น attribute เพิ่มเติมนอกเหนือจาก owner และ balance ที่รับมาจาก parent
  - มี method apply_interest() ที่คำนวณดอกเบี้ยจาก balance ปัจจุบันแล้วเพิ่มเข้าบัญชีโดยอัตโนมัติ
  - มีข้อจำกัดพิเศษสำหรับการถอนเงิน คือต้องคงเหลือยอดขั้นต่ำ 500 บาทในบัญชีเสมอหลังถอน ซึ่งแตกต่างจากบัญชีทั่วไปที่ถอนได้จนเหลือ 0

Live Coding: BankAccount → SavingsAccount

```
savings = SavingsAccount("Alice", 10000.0, 0.05)
savings.display_info()

savings.deposit(5000.0)
savings.withdraw(2000.0)
savings.apply_interest()
savings.display_info()

print(isinstance(savings, SavingsAccount))
print(isinstance(savings, BankAccount))
```



ผลลัพธ์ที่คาดหวัง

```
[SavingsAccount] เจ้าของ: Alice | ยอดเงิน: 10000.00 บาท | อัตราดอกเบี้ย: 5.0%
ฝากเงิน 5000.00 บาท | ยอดคงเหลือ: 15000.00 บาท
ถอนเงิน 2000.00 บาท | ยอดคงเหลือ: 13000.00 บาท
ดอกเบี้ย 5.0% = 650.00 บาท | ยอดใหม่: 13650.00 บาท
[SavingsAccount] เจ้าของ: Alice | ยอดเงิน: 13650.00 บาท | อัตราดอกเบี้ย: 5.0%
True
True
```

จุดที่ควรสังเกตก่อนเริ่มทำ

- จุดแรกคือ withdraw() ของ SavingsAccount มีพฤติกรรมต่างจาก parent ซึ่งหมายความว่าต้อง override ไม่ใช่เพิ่ม method ใหม่
- จุดที่สองคือ display_info() ก็ต้อง override เช่นกันเพื่อแสดง interest_rate เพิ่มเติม
- ทั้งสองจุดนี้คือหัวใจของตัวอย่างนี้ที่แสดงให้เห็นว่า Inheritance ไม่ได้หมายถึงการรับทุกอย่างมาแบบตายตัว แต่ child class สามารถปรับพฤติกรรมที่ต้องการได้อย่างอิสระ

```
def withdraw(self, amount: float) -> None:
    if amount <= 0:
        print("จำนวนเงินต้องมากกว่า 0")
        return
    if amount > self.balance:
        print("ยอดเงินไม่เพียงพอ")
        return
    self.balance -= amount
    print(f"ถอนเงิน {amount:.2f} บาท | ยอดคงเหลือ: {self.balance:.2f} บาท")

def display_info(self) -> None:
    print(f"[BankAccount] เจ้าของ: {self.owner} | ยอดเงิน: {self.balance:.2f} บาท")
```

ทบทวนคลาสแม่ก่อนเริ่ม

- SavingsAccount จะ inherit จาก BankAccount เวอร์ชัน Encapsulation ที่มีโครงสร้างดังนี้

```
class BankAccount:

    def __init__(self, account_number, owner, balance=0):…


    @property
    def account_number(self):…

    @property
    def owner(self):…

    @property
    def balance(self):…

    @property
    def transaction_count(self):…

    def deposit(self, amount):…

    def withdraw(self, amount):…

    def display_info(self):…
```

จุดสำคัญที่ต้องจำไว้ตลอดโจทย์นี้คือ balance ไม่มี setter และ __balance ถูกซ่อนอยู่ในคลาสแม่

ดังนั้น SavingsAccount จะไม่สามารถแก้ไข balance โดยตรงได้เลย ไม่ว่าจะพยายามด้วยวิธีใด

ส่วนที่ 1 — วิเคราะห์ก่อนเขียนโค้ด

- ทดสอบ is-a test ก่อนเสมอ
  - SavingsAccount เป็น BankAccount ประเภทหนึ่งหรือไม่?
    - คำตอบคือใช่ เพราะบัญชีออมทรัพย์มีคุณสมบัติพื้นฐานของบัญชีธนาคารครบถ้วน
    - และมีพฤติกรรมเพิ่มเติม จึงผ่าน is-a test และเหมาะสมที่จะใช้ Inheritance
- สิ่งที่ได้มาฟรีจากการ inherit คือ account_number, owner, balance, transaction_count
  - ทั้ง 4 properties และ deposit() ที่ไม่ต้องเขียนใหม่เลย
- สิ่งที่ต้องเพิ่มเข้ามาใหม่ คือ
  - attribute interest_rate
  - method apply_interest()
- สิ่งที่ต้อง override คือ
  - withdraw() เพราะบัญชีออมทรัพย์มีเงื่อนไขพิเศษที่ต่างจาก parent
  - display_info() เพราะต้องแสดง interest_rate เพิ่มเติม

การเขียน class เพื่อสร้างความสัมพันธ์ subclass

```python
from lecture2 import BankAccount

class SavingsAccount(BankAccount):
```



รู้จัก Attribute ใหม่ก่อนลงมือเขียน

SavingsAccount มี attribute เพิ่มจาก parent เพียงตัวเดียว

- _interest_rate เก็บเป็น float คือ อัตราดอกเบี้ยต่อปีในรูปทศนิยม
  - เช่น 0.03 หมายถึง 3% มีช่วงค่าที่ถูกต้องคือ 0.0 ถึง 0.10 (0% ถึง 10%)
  - เปลี่ยนแปลงได้แต่ต้องผ่านการตรวจสอบก่อนเสมอ
- attribute ที่เหลือทั้งหมด (_account_number, _owner, _balance, _transaction_count) เป็นของ parent อยู่แล้ว
  - SavingsAccount ไม่ต้องประกาศซ้ำ
  - และที่สำคัญคือเข้าถึง _balance ตรงๆ ไม่ได้ด้วย

ส่วนที่ 2 — Constructor

```
def __init__(self, account_number, owner, balance, interest_rate):
```

- บรรทัดแรกสุดต้องเป็น super().__init__(account_number, owner, balance) เสมอ
  - เพื่อให้ BankAccount เตรียม _account_number, _owner, _balance, _transaction_count ให้ครบก่อน
- หลังจากนั้นจึงกำหนด self.interest_rate = interest_rate ผ่าน setter ไม่ใช่ self._interest_rate = interest_rate โดยตรง
  - เพื่อให้ validation ทำงานตั้งแต่ตอนสร้าง object เช่นเดียวกับหลักการที่ใช้กับ balance ใน BankAccount

ส่วนที่ 3 — Properties

- interest_rate — property + setter ตรวจสองเงื่อนไขตามลำดับ คือ
  - ต้องเป็น int หรือ float หากไม่ใช่ raise TypeError("อัตราดอกเบี้ยต้องเป็นตัวเลข")
  - และต้องอยู่ในช่วง 0.0 ถึง 0.10 หากไม่ใช่ raise ValueError("อัตราดอกเบี้ยต้องอยู่ในช่วง 0.0 ถึง 0.10")
- ไม่ต้องเขียน property สำหรับ account_number, owner, balance, transaction_count ซ้ำ เพราะ inherit มาจาก parent โดยอัตโนมัติแล้ว

ส่วนที่ 4 — Methods (1)

apply_interest()

- คำนวณดอกเบี้ยจาก self.balance * self.interest_rate
- จากนั้นต้องเพิ่มเข้าบัญชีผ่าน self.deposit(interest) เท่านั้น
- ห้ามเขียน self._balance += interest หรือ self.balance += interest เด็ดขาด เพราะ _balance เป็นของ parent ซึ่ง SavingsAccount มองไม่เห็น และ balance ก็ไม่มี setter ด้วย
- การเรียก self.deposit() จึงเป็นทางเดียวที่ถูกต้อง และยังได้ประโยชน์แถมคือ transaction_count จะถูกนับให้อัตโนมัติผ่าน deposit() ที่มีอยู่แล้ว

ส่วนที่ 4 — Methods (2)

withdraw(amount)

- ต้องตรวจสอบเงื่อนไขพิเศษก่อนว่ายอดคงเหลือหลังถอนต้องไม่ต่ำกว่า 500 บาท หากไม่ผ่าน raise ValueError("ยอดคงเหลือต้องไม่ต่ำกว่า 500 บาท")
- เมื่อผ่านเงื่อนไขนี้แล้ว ต้องเรียก super().withdraw(amount) เพื่อให้ parent จัดการ validation พื้นฐาน (amount ต้องมากกว่า 0, ไม่เกิน balance) และการหักเงินจริงให้
- ห้ามเขียน logic การถอนซ้ำเองทั้งหมด เพราะจะทำให้เกิดโค้ดซ้ำซ้อนและเสี่ยง bug หากภายหลังต้องแก้ logic การถอนของ parent

ส่วนที่ 4 — Methods (3)

display_info() — Override

- ต้องแสดงข้อมูลทั้งหมดเหมือน parent บวกกับ interest_rate ต่อท้าย
- วิธีที่ถูกต้องคือเรียก super().display_info() ก่อน
- แล้วจึง print() ข้อมูลส่วนที่เพิ่มเติม ไม่ใช่คัดลอก logic การแสดงผลของ parent มาเขียนซ้ำทั้งหมด

ส่วนที่ 5 — ผลลัพธ์ที่คาดหวัง

```
acc2 = SavingsAccount("SAV-0001", "Nana", 10000, 0.03)
acc2.display_info()

# properties ที่ inherit มาจาก parent — ใช้งานได้ทันทีโดยไม่ต้องเขียนใหม่
print(acc2.account_number)     # SAV-0001
print(acc2.owner)               # Nana
print(acc2.balance)             # 10000
print(acc2.transaction_count)   # 0

# property ใหม่ของ SavingsAccount
print(acc2.interest_rate)       # 0.03

# apply_interest() — method ใหม่
acc2.apply_interest()
print(acc2.balance)             # 10300.0
print(acc2.transaction_count)   # 1  ← เพิ่มผ่าน deposit() อัตโนมัติ
```

ส่วนที่ 5 — ผลลัพธ์ที่คาดหวัง (2)

```
# withdraw() ที่ถูก override — เงื่อนไขยอดขั้นต่ำ 500
try:
    acc2.withdraw(9000)
except ValueError as e:
    print(f"แจ้งเตือน: {e}")
# → แจ้งเตือน: ยอดคงเหลือต้องไม่ต่ำกว่า 500 บาท
try:
    acc2.withdraw(8000)
except ValueError as e:
    print(f"แจ้งเตือน: {e}")
print(acc2.balance)              # 1300.0
print(acc2.transaction_count)    # 2

# display_info() ที่ถูก override
acc2.display_info()
```

ส่วนที่ 5 — ผลลัพธ์ที่คาดหวัง (3)

```
# deposit() ที่ inherit มา — ไม่ต้อง override เพราะพฤติกรรมเหมือน parent ทุกประการ
acc2.deposit(1000)
print(acc2.balance)             # 2300.0

# การป้องกันที่ inherit มาจาก parent ยังทำงานอยู่
try:
    acc2.balance = 999999
except AttributeError as e:
    print(f"แจ้งเตือน: {e}")
# → แจ้งเตือน: property 'balance' of 'SavingsAccount' object has no setter

try:
    acc2.interest_rate = 0.5
except ValueError as e:
    print(f"แจ้งเตือน: {e}")
# → แจ้งเตือน: อัตราดอกเบี้ยต้องอยู่ในช่วง 0.0 ถึง 0.10

# ทดสอบ constructor validation
try:
    acc3 = SavingsAccount("SAV-0002", "Bob", 5000, 0.5)
except ValueError as e:
    print(f"แจ้งเตือน: {e}")
# → แจ้งเตือน: อัตราดอกเบี้ยต้องอยู่ในช่วง 0.0 ถึง 0.10
```

Common Mistakes

ผิดพลาดที่ 1: ลืมเรียก super().__init__()

ผิด: ลืมเรียก super().__init__()

```
class SavingsAccount(BankAccount):
    def __init__(self, owner: str, balance: float, interest_rate: float):
        self._interest_rate = interest_rate   # ← parent attributes ไม่ถูกสร้าง

savings = SavingsAccount("Alice", 1000.0, 0.05)
savings.deposit(500.0)   # AttributeError: '_BankAccount__balance' ไม่มี
```

ถูก

```
class SavingsAccount(BankAccount):
    def __init__(self, owner: str, balance: float, interest_rate: float):
        super().__init__(owner, balance)      # ← สร้าง parent attributes ก่อน
        self._interest_rate = interest_rate
```

ผิดพลาดที่ 2: เรียก super().__init__() ในลำดับผิด

ผิด

```python
class SavingsAccount(BankAccount):
    def __init__(self, owner: str, balance: float, interest_rate: float):
        self._interest_rate = interest_rate
        self.apply_interest()                # ← ใช้ self.balance ก่อนที่ parent สร้าง!
        super().__init__(owner, balance)     # ← สายเกินไป
```

กฎปฏิบัติ: super().__init__() ควรเป็นบรรทัดแรกของ child's __init__ เสมอ ยกเว้นมีเหตุผลชัดเจนที่ต้องทำก่อน

ผิดพลาดที่ 3: ส่ง argument ผิดจำนวนให้ super().__init__()

ผิด :ลืมส่ง balance ไปด้วย

```python
class SavingsAccount(BankAccount):
    def __init__(self, owner: str, balance: float, interest_rate: float):
        super().__init__(owner)      # ← ลืม balance → balance จะเป็น 0.0 เสมอ
        self._interest_rate = interest_rate
```

ให้ดู signature ของ parent's __init__ ก่อนเรียก super() เสมอ

ผิดพลาดที่ 4: override method โดยลืมเรียก super() ทั้งที่ควรเรียก

ผิด

```
# อาจเป็นปัญหา — override display_info แต่ไม่ดึง logic เดิมมาเลย
class SavingsAccount(BankAccount):
    def display_info(self) -> None:
        # เขียนใหม่ทั้งหมดโดยลืม owner และ balance
        print(f"ดอกเบี้ย: {self._interest_rate * 100:.1f}%")
```

ถ้าต้องการ behavior เดิมและเพิ่มส่วนใหม่ ให้ใช้ super().display_info() แล้วต่อด้วยส่วนที่เพิ่มขึ้น

ผิดพลาดที่ 5: Inherit เมื่อไม่ผ่าน is-a test

ผิด — Customer ไม่ใช่ BankAccount

```
class Customer(BankAccount):
    pass
```

Customer IS-A BankAccount? ไม่ใช่!

ถูก — Customer HAS-A BankAccount ใช้ composition

```
class Customer:
    def __init__(self, name: str):
        self.name = name
        self.account = BankAccount(name)
```

← เก็บ BankAccount ไว้ข้างใน



Recap

- เมื่อ class หลายตัวมีลักษณะร่วมกัน Inheritance คือ กลไกที่ช่วยให้ต่อยอดโค้ดได้อย่างมีระบบ โดยไม่ต้องเขียนซ้ำในทุก class
- ตัดสินใจใช้ Inheritance ด้วย is-a test
  - หาก child class "เป็น" parent class ได้อย่างสมเหตุสมผล Inheritance จึงเหมาะสม
  - ไม่เช่นนั้น ควรใช้ Composition แทน เพราะ Inheritance ที่ไม่ผ่าน is-a test นำไปสู่ Inappropriate Intimacy ซึ่งทำให้ระบบเปราะบางและดูแลรักษาได้ยาก
- เมื่อใช้ Inheritance แล้ว super().__init__() คือสิ่งแรกที่ child class ต้องเรียกเสมอ
  - เพื่อให้ parent มีโอกาสเตรียม attribute ของตัวเองก่อนที่ child จะเพิ่ม attribute หรือ logic ของตัวเอง
- Method Override คือกลไกที่ให้ child class ปรับพฤติกรรมเฉพาะที่แตกต่างจาก parent ได้
  - โดยยังคงใช้ประโยชน์จาก method เดิมผ่าน super() ได้เมื่อจำเป็น
- Inheritance ไม่ใช่เครื่องมือสำหรับ "นำโค้ดกลับมาใช้ซ้ำ" เท่านั้น แต่คือการสร้าง class hierarchy ที่สะท้อนความสัมพันธ์ที่เป็นจริงของสิ่งที่กำลังจำลอง
- การเลือกใช้ให้ถูกบริบทคือทักษะด้านการออกแบบที่สำคัญกว่าการจำ syntax



THANK YOU!